

# A GENETIC ALGORITHM APPROACH TO ADAPTIVE PACKET SIZE CONTROL FOR UDP



**Department of Computer Science and Engineering**  
Khulna University of Engineering & Technology (KUET)

**CSE 4106 : Computer Networks Laboratory**

**Submitted to:**

Md. Sakhawat Hossain  
Lecturer, Dept. of CSE, KUET

Farhan Sadaf  
Lecturer, Dept. of CSE, KUET

**Submitted by:**

Tirtho Mondal  
Roll 2007117  
4th Year, 1st Semester

# Contents

|          |                                             |          |
|----------|---------------------------------------------|----------|
| <b>1</b> | <b>Abstract</b>                             | <b>2</b> |
| <b>2</b> | <b>Introduction</b>                         | <b>3</b> |
| 2.1      | Motivation and Problem Context . . . . .    | 3        |
| 2.2      | Aim of the Project . . . . .                | 3        |
| 2.3      | Objectives . . . . .                        | 3        |
| 2.4      | Scope and Significance . . . . .            | 3        |
| <b>3</b> | <b>Background Study</b>                     | <b>3</b> |
| 3.1      | UDP Characteristics . . . . .               | 3        |
| 3.2      | Congestion Control Mechanisms . . . . .     | 4        |
| 3.3      | Fundamentals of Genetic Algorithm . . . . . | 4        |
| <b>4</b> | <b>System Design and Architecture</b>       | <b>4</b> |
| 4.1      | Network Topology . . . . .                  | 4        |
| 4.2      | Sender Node . . . . .                       | 5        |
| 4.3      | Router Nodes . . . . .                      | 6        |
| 4.4      | Receiver Node . . . . .                     | 8        |
| <b>5</b> | <b>Overall Workflow</b>                     | <b>8</b> |
| <b>6</b> | <b>Discussion</b>                           | <b>9</b> |
| <b>7</b> | <b>Conclusion</b>                           | <b>9</b> |

## List of Figures

|   |                                                         |   |
|---|---------------------------------------------------------|---|
| 1 | Architecture of GA-based adaptive UDP network. . . . .  | 5 |
| 2 | Initial Stage of UDP Packet Transmission . . . . .      | 5 |
| 3 | Feedback Flow to the Sender . . . . .                   | 6 |
| 4 | The update packet size is sent by the sender. . . . .   | 8 |
| 5 | The update packet size is sent to the receiver. . . . . | 8 |

# 1 Abstract

UDP (User Datagram Protocol) is known for its speed and efficiency in transmitting real-time data but lacks essential features such as congestion control, reliability, and adaptability, often resulting in packet loss and reduced network performance during congestion. To overcome these limitations, this study introduces an intelligent adaptive packet size control mechanism based on a Genetic Algorithm (GA). The GA operates at the router level, dynamically adjusting packet size recommendations according to a computed Congestion Index (CI) that reflects the network's real-time state. Sender nodes adapt their transmission sizes based on this feedback, forming a self-regulating, congestion-aware UDP framework. Implemented in the OMNeT++ simulation environment using C++ for module definitions and evolutionary logic, the proposed approach demonstrates that GA-based adaptive control enhances throughput stability, minimizes packet loss, and maintains consistent network performance under varying load conditions. These findings highlight the effectiveness of bio-inspired learning techniques in improving traditional non-congestion-aware protocols like UDP, bridging the gap between transmission efficiency and adaptive network behavior.

## **2 Introduction**

### **2.1 Motivation and Problem Context**

With the increasing use of multimedia services, IoT devices, and real-time communication systems, the demand for fast and low-latency network protocols has grown significantly. UDP is a popular choice for time-sensitive applications like video streaming, online gaming, and VoIP because it offers minimal overhead and quick data transmission. However, its lack of congestion control makes it vulnerable to instability when network traffic is heavy. As multiple UDP senders transmit at high speeds, routers can become congested, causing queue overflows and packet loss. Unlike TCP, which automatically slows down during congestion, UDP continues sending data at a constant rate and packet size, intensifying network congestion. This limitation highlights the need for an adaptive mechanism that can intelligently regulate UDP's transmission behavior under varying network conditions without changing its core design.

### **2.2 Aim of the Project**

This project aims to design and evaluate a Genetic Algorithm (GA)-based adaptive control system that dynamically adjusts UDP packet sizes according to real-time network congestion feedback. By applying evolutionary principles, the system continuously evolves optimal transmission parameters, ensuring efficient and stable performance under varying network conditions.

### **2.3 Objectives**

- To optimize packet size dynamically based on real-time network conditions.
- To reduce packet loss and congestion in varying network environments.
- To improve overall data transmission efficiency and throughput.
- To simulate and analyze adaptive behavior using the OMNeT++ network simulator.

### **2.4 Scope and Significance**

Unlike conventional TCP congestion control, which relies on window scaling and acknowledgments, this GA-based approach focuses purely on packet size adaptation making it lightweight, scalable, and suitable for connectionless environments. The approach can be generalized for IoT, sensor networks, and other real-time systems.

## **3 Background Study**

### **3.1 UDP Characteristics**

The User Datagram Protocol (UDP) is a connectionless transport layer protocol that enables rapid data transmission with minimal overhead. It does not establish a connection or guarantee reliable delivery, making it suitable for applications where speed and efficiency are prioritized over reliability, such as real-time streaming, gaming, and VoIP.

## 3.2 Congestion Control Mechanisms

Congestion control in UDP using Genetic Algorithm (GA) aims to manage network traffic efficiently, since UDP itself lacks built-in congestion control. The Genetic Algorithm is applied to optimize parameters such as packet sending rate and loss recovery based on real-time network feedback. By evolving the best transmission strategies through selection, crossover, and mutation, GA helps reduce packet loss, minimize delay, and improve overall throughput, providing an intelligent and adaptive solution for congestion control in UDP-based systems.

## 3.3 Fundamentals of Genetic Algorithm

1. **Initialization:** A population of random candidate solutions (called *chromosomes*) is generated.
2. **Fitness Evaluation:** Each solution is evaluated using a fitness function that measures how well it solves the problem.
3. **Selection:** The best-performing solutions are chosen as parents for reproduction, mimicking “survival of the fittest.”
4. **Crossover:** Two parent chromosomes exchange parts of their genetic information to create offspring, introducing new combinations.
5. **Mutation:** Small random changes are applied to some offspring to maintain diversity and avoid premature convergence.
6. **Replacement:** The new generation replaces the old one, and the process repeats until a stopping condition (like reaching an optimal solution) is met.

GA’s adaptability and robustness make it ideal for optimizing parameters in dynamic, non-linear systems such as communication networks.

# 4 System Design and Architecture

## 4.1 Network Topology

The simulated network (Figure 1) includes:

1. Three *SenderNodes* transmitting UDP-like packets.
2. Two *RouterNodes* performing packet forwarding and GA-based adaptation.
3. Two *ReceiverNodes* collecting packets for performance measurement.

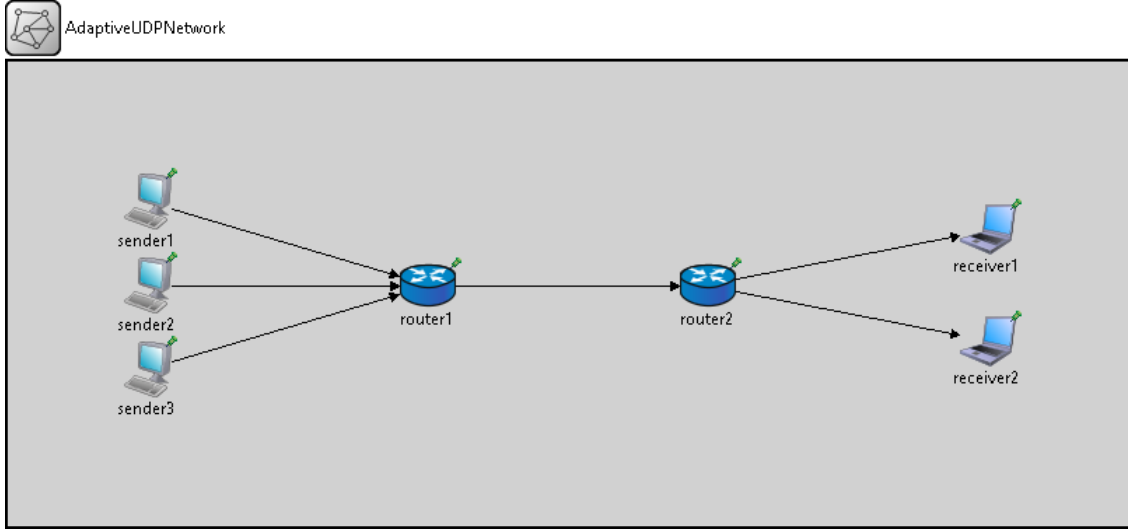


Figure 1: Architecture of GA-based adaptive UDP network.

The goal is to simulate adaptive communication, where senders adjust packet size based on congestion feedback, and routers evolve routing and transmission strategies using a genetic algorithm.

## 4.2 Sender Node

The sender node generates packets at regular intervals and adjusts their size based on feedback from the router. The packet generation process is given by

$$P_k = \text{Packet of size } S_k \text{ bytes sent at time } t_k$$

where

$$t_k = k \times T_s, \quad T_s = 0.2 \text{ seconds.} \quad (1)$$

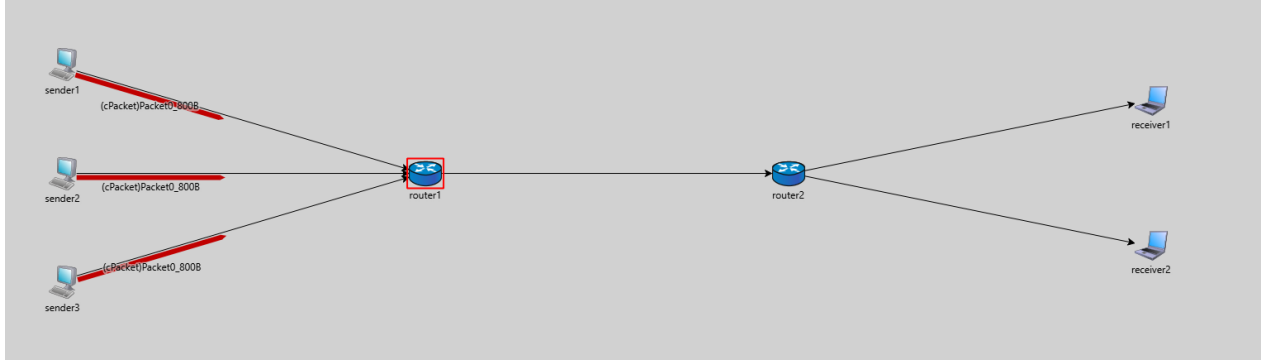


Figure 2: Initial Stage of UDP Packet Transmission

Based on the congestion index (CI), the packet size is updated as

$$S_{k+1} = \begin{cases} \min(1500, S_k + 100), & \text{if } CI < 0.3 \\ S_k, & \text{if } 0.3 \leq CI < 0.6 \\ \max(400, S_k - 150), & \text{if } 0.6 \leq CI < 0.8 \\ \max(200, S_k - 300), & \text{if } CI \geq 0.8 \end{cases} \quad (2)$$

When congestion is low, packet size increases to use more bandwidth. At medium congestion, it remains constant, and at high congestion, it decreases to reduce network load.

### 4.3 Router Nodes

The Genetic Algorithm (GA) within the Router Node model evolves optimal routing and packet-size strategies by iteratively refining a population of candidate solutions. Each individual in the population represents a potential routing configuration, and the process involves initialization, fitness evaluation, selection, crossover, mutation, adaptive control, and replacement until convergence is achieved.

**1. Initialization** At the start of the genetic algorithm process, an initial population of  $P$  individuals is generated randomly. Each individual is represented as

$$I_i = \{\text{path}_i, \text{packetSize}_i, \text{fitness}_i\} \quad (3)$$

where  $\text{path}_i$  denotes the routing path index assigned to the  $i^{\text{th}}$  individual,  $\text{packetSize}_i$  represents the packet size (in bytes), and  $\text{fitness}_i$  corresponds to the evaluated performance of that individual.

In this stage, key control parameters of the algorithm are also initialized: the mutation rate  $m$ , which defines the probability of introducing random variations into offspring; the crossover rate  $c$ , which determines the likelihood of recombination between selected parents; and the population size  $P$ , specifying the number of individuals maintained per generation. Additionally, the

**2. Fitness Evaluation** Each individual's performance is evaluated through a fitness function designed to minimize congestion and packet loss. The fitness value is defined as

$$f = 1 - CI \quad (4)$$

where the congestion index  $CI$  is computed as

$$CI = 0.7 \times \frac{Q}{C} + 0.3 \times \frac{D}{D+1} \quad (5)$$

Here,  $Q$  represents the current queue length or buffer occupancy,  $C$  denotes the maximum channel capacity, and  $D$  indicates the packet drop rate or packet loss ratio. A lower congestion index implies better network performance, resulting in a higher fitness score  $f$ .

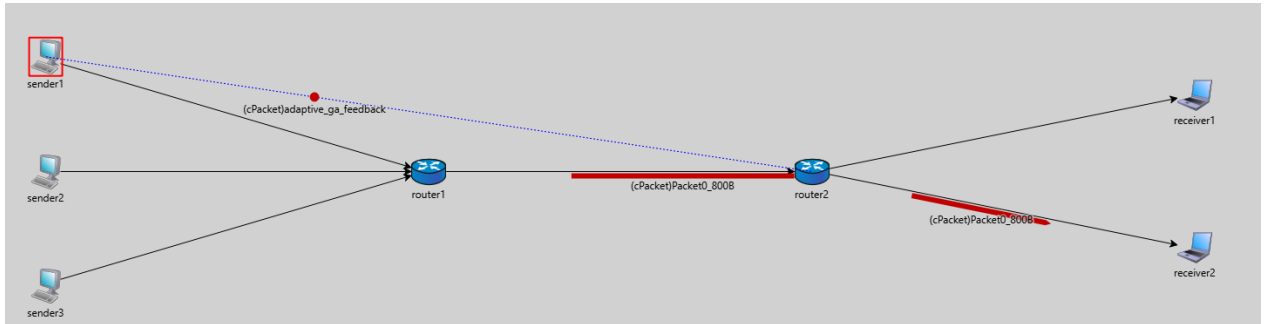


Figure 3: Feedback Flow to the Sender

**3. Selection** During selection, individuals with higher fitness values have a greater chance of being chosen as parents for the next generation. This probabilistic selection mechanism ensures that well-performing routing strategies are more likely to reproduce, simulating the principle of “survival of the fittest.”

**4. Crossover (Recombination)** Once parents are selected, they exchange parts of their chromosomes to create offspring. In the simplest case, a one-point crossover can be expressed as

$$\text{child.path} = \text{parent2.path}, \quad \text{child.packetSize} = \text{parent1.packetSize}. \quad (6)$$

The crossover probability  $c$  determines how frequently this recombination occurs. Through crossover, new combinations of routing paths and packet sizes are produced, promoting genetic diversity and enabling exploration of potentially better configurations.

**5. Mutation** Mutation introduces small random variations into the offspring with a probability defined by the mutation rate  $m$ . This operation can be expressed as

$$\text{child.path} \leftarrow \text{random}(0, \text{gateSize} - 1), \quad \text{child.packetSize} \leftarrow \text{random}(400, 1500).$$

By altering these parameters randomly, the GA avoids premature convergence and ensures a broad exploration of the search space.

**6. Parameter Adjustment (Adaptive Control)** The GA dynamically adjusts the mutation and crossover rates based on the performance of recent generations. The mutation rate is updated according to

$$m_{\text{new}} = \begin{cases} 1.1 \times m, & \text{if improvement} < 0.01 \\ 0.95 \times m, & \text{otherwise} \end{cases} \quad (7)$$

and the crossover rate is modified as

$$m_{\text{new}} = 1.2 \times m, \quad c_{\text{new}} = 0.9 \times c \quad (\text{if diversity decreases}), \quad \text{else} \quad c_{\text{new}} = 1.1 \times c. \quad (8)$$

Here, the term “improvement” refers to the relative change in average fitness between generations, while “diversity” quantifies the statistical variation among individuals. These adaptive updates help maintain balance between exploration and exploitation as network conditions evolve.

**7. Replacement** After crossover and mutation, the newly created offspring replace the least-fit individuals in the population. This replacement step ensures that new solutions have the opportunity to propagate through successive generations, while weaker ones are gradually eliminated.

**8. Termination** The GA continues iterating through generations until a termination condition is met. Common stopping criteria include reaching a predefined number of generations, observing negligible fitness improvement between successive iterations, or detecting stabilized network performance that indicates convergence.





Figure 4: The update packet size is sent by the sender.



Figure 5: The update packet size is sent to the receiver.

#### 4.4 Receiver Node

The receiver node receives packets from the router and maintains a count of successfully delivered packets. The total number of packets received is represented as

$$N_r = \text{TotalPacketsReceived} \quad (9)$$

This value reflects the overall effectiveness of the transmission process and helps evaluate network performance.

### 5 Overall Workflow

The overall communication process operates as follows:

1. The sender node transmits packets at regular intervals.
2. The router node receives and stores packets in its buffer, possibly dropping some if the buffer is full.
3. The router applies a genetic algorithm to optimize routing paths and packet-size strategies.
4. The router calculates the congestion index and sends feedback to the sender nodes.
5. The sender nodes adjust their packet size based on the received feedback.
6. The receiver node logs all successfully received packets.

## 6 Discussion

The proposed GA mechanism introduced an intelligent, decentralized control layer without altering UDP headers or requiring extra control channels. By learning from congestion feedback, the system achieved dynamic equilibrium between packet efficiency and network stability. Unlike heuristic control schemes, GA-based adaptation exhibits resilience under fluctuating conditions. The probabilistic nature of GA allows continuous exploration, preventing stagnation even in steady-state networks.

## 7 Conclusion

This work demonstrated that Genetic Algorithms can effectively enable congestion-aware and adaptive packet size control in UDP. The OMNeT++ simulations showed that GA-based adaptation reduces packet loss, stabilizes throughput, and maintains fairness among senders, confirming that evolutionary methods can be integrated into network protocols with minimal computational overhead.

Future efforts may focus on extending the GA to optimize additional parameters such as transmission intervals and queue thresholds, or on combining it with reinforcement learning for hybrid adaptive control. Further exploration in wireless or IoT environments, as well as developing multi-objective GA models to balance throughput, delay, and energy efficiency, could enhance its real-world applicability.

## References

- [1] H. Lingaraj, “A study on genetic algorithm and its applications,” *International Journal of Computer Sciences and Engineering*, vol. 4, pp. 139–143, 10 2016.
- [2] S. D. Immanuel and U. K. Chakraborty, “Genetic algorithm: An approach on optimization,” in *2019 International Conference on Communication and Electronics Systems (ICCES)*, 2019, pp. 701–708.
- [3] Q. Sun and H. Li, “Research and application of a udp-based reliable data transfer protocol in wireless data transmission,” in *2011 International Conference on Computer Science and Service System (CSSS)*, 2011, pp. 1514–1516.
- [4] S. Sunassee, A. Mungur, S. Armoogum, and S. Pudaruth, “A comprehensive review on congestion control techniques in networking,” in *2021 5th International Conference on Computing Methodologies and Communication (ICCMC)*, 2021, pp. 305–312.

Table 1: Main Initialization Parameters (Appendix A)

| Component       | Parameter                   | Initial Value    | Description                                                          |
|-----------------|-----------------------------|------------------|----------------------------------------------------------------------|
| Simulation      | sim-time-limit              | <b>100 s</b>     | Total duration of the simulation run.                                |
| Simulation      | record-eventlog             | <b>true</b>      | Enables detailed event logging for later analysis.                   |
| All Modules     | capacity                    | <b>100</b>       | Queue capacity for packet buffering in routers.                      |
| SenderNode      | packetSize                  | <b>800 bytes</b> | Initial packet size before genetic adaptation.                       |
| SenderNode      | sendInterval                | <b>0.2 s</b>     | Time interval between packet transmissions.                          |
| RouterNode (GA) | mutationRate                | <b>0.12</b>      | Initial mutation probability for the genetic algorithm evolution.    |
| RouterNode (GA) | crossoverRate               | <b>0.8</b>       | Initial crossover probability used in population evolution.          |
| RouterNode (GA) | population size             | <b>10</b>        | Number of candidate individuals maintained by the genetic algorithm. |
| Network Links   | sender → router delay       | <b>1 ms</b>      | Link transmission delay between sender nodes and router1.            |
| Network Links   | sender → router datarate    | <b>10 Mbps</b>   | Transmission data rate for sender-to-router links.                   |
| Network Links   | router1 → router2 delay     | <b>2 ms</b>      | Delay between the intermediate routers.                              |
| Network Links   | router1 → router2 datarate  | <b>5 Mbps</b>    | Data rate for the router-to-router link.                             |
| Network Links   | router2 → receiver delay    | <b>2 ms</b>      | Delay on the final hop to receivers.                                 |
| Network Links   | router2 → receiver datarate | <b>8 Mbps</b>    | Bandwidth on the last hop to receivers.                              |

Table 2: Summary of Core Equations (Appendix B)

| Concept                               | Equation                                                                                                                                                                                                                            | Description                                                                                                  |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <b>Packet Generation Time</b>         | $t_k = k \times T_s, \quad T_s = 0.2 \text{ s}$                                                                                                                                                                                     | Defines when each packet is generated by the sender node at fixed intervals.                                 |
| <b>Congestion Index (CI)</b>          | $CI = 0.7 \times \frac{Q}{C} + 0.3 \times \frac{D}{D+1}$                                                                                                                                                                            | Computes real-time congestion level from queue length $Q$ , capacity $C$ , and dropped packets $D$ .         |
| <b>Fitness Function</b>               | $f = 1 - CI$                                                                                                                                                                                                                        | Measures how effectively a routing individual minimizes congestion; higher fitness means better performance. |
| <b>Packet Size Adaptation Rule</b>    | $S_{k+1} = \begin{cases} \min(1500, S_k + 100), & \text{if } CI < 0.3 \\ S_k, & \text{if } 0.3 \leq CI < 0.6 \\ \max(400, S_k - 150), & \text{if } 0.6 \leq CI < 0.8 \\ \max(200, S_k - 300), & \text{if } CI \geq 0.8 \end{cases}$ | Updates packet size dynamically at the sender based on congestion feedback from routers.                     |
| <b>Crossover Operation</b>            | child.path = parent2.path,    child.packetSize = parent1.packetSize                                                                                                                                                                 | Combines traits from two parents to create a new individual in the GA population.                            |
| <b>Mutation Operation</b>             | child.path $\leftarrow$ random(0, gateSize - 1),    child.packetSize $\leftarrow$ random(400, 1500)                                                                                                                                 | Introduces random variations to maintain diversity and avoid premature convergence.                          |
| <b>Mutation Rate Adjustment</b>       | $m_{\text{new}} = \begin{cases} 1.1 \times m, & \text{if improvement} < 0.01 \\ 0.95 \times m, & \text{otherwise} \end{cases}$                                                                                                      | Adapts mutation probability according to the rate of improvement between generations.                        |
| <b>Adaptive Control of Parameters</b> | if diversity decreases: $m_{\text{new}} = 1.2m, c_{\text{new}} = 0.9c$ ;    else: $c_{\text{new}} = 1.1c$                                                                                                                           | Adjusts GA parameters dynamically based on diversity to balance exploration and exploitation.                |
| <b>Receiver Metric</b>                | $N_r = \text{TotalPacketsReceived}$                                                                                                                                                                                                 | Counts the total successfully received packets, used to evaluate overall network performance.                |